

Simulación de Entrevista Técnica — Redis

Guía de estudio en formato entrevista · uso de Redis en el sistema POS (rate limiting en el gateway · caché · sesiones · denylist de JWT)

Guía de estudio en formato entrevista centrada en **Redis** dentro de este proyecto POS. Cada sección incluye la **pregunta del entrevistador**, una **respuesta modelo** y, cuando aplica, **preguntas de seguimiento** (follow-ups).

Formato sugerido: 40–50 min. Bloques: fundamentos (10'), estructuras de datos (10'), rate limiting en el gateway (10'), caché y sesiones (10'), operación/HA (10').

0. Warm-up — Redis en el proyecto

P ¿Para qué se usa Redis en este sistema POS?

R Redis (:6379) cumple sobre todo dos roles: es el **store de rate limiting** del API Gateway (Spring Cloud Gateway usa un `RequestRateLimiter` basado en Redis con algoritmo token-bucket) y sirve como **caché** de baja latencia. Además es el candidato natural para casos transversales como **denylist de JWT** (revocación de tokens antes de expirar) y almacenamiento de sesiones o datos efímeros. Es un almacén **en memoria** clave-valor, single-threaded para comandos, lo que lo hace extremadamente rápido y predecible.

1. Fundamentos

P1 ¿Por qué Redis es tan rápido si es single-threaded?

R Porque trabaja **en memoria** (RAM), evita el costo de I/O de disco en la ruta caliente, y su modelo **single-threaded** para la ejecución de comandos elimina bloqueos y contención de locks: cada comando es atómico y se procesa secuencialmente sobre un event loop con multiplexación de I/O (epoll/kqueue). Redis 6+ sí usa hilos auxiliares para I/O de red, pero la lógica de datos sigue siendo un solo hilo, lo que simplifica el modelo de consistencia.

FOLLOW-UP ¿Qué riesgo tiene un comando lento como `KEYS *` ? — Bloquea el único hilo y frena a todos los clientes. En producción se usa `SCAN` (cursor incremental) en su lugar.

P2 ¿Redis vs Memcached para caché?

R Memcached es puramente caché clave-valor multihilo, muy simple. Redis ofrece **estructuras de datos ricas** (listas, sets, hashes, sorted sets, streams), **persistencia** opcional (RDB/AOF), replicación, scripting Lua y pub/sub. Para este proyecto Redis gana porque el rate limiting con token bucket necesita operaciones atómicas y expiración, algo que Redis resuelve de forma nativa.

2. Estructuras de datos

P3 ¿Qué estructuras de datos ofrece Redis y para qué las usarías aquí?

R

- **String**: contadores de rate limit, caché de respuestas JSON, flags.
- **Hash**: representar un objeto (p. ej. datos de sesión) con campos.
- **List**: colas simples, logs recientes.
- **Set**: denylist de JWT revocados (pertenencia $O(1)$).
- **Sorted Set**: rate limiting por ventana deslizante (score = timestamp), rankings.
- **Stream**: event log con consumer groups (alternativa ligera a Kafka).

FOLLOW-UP ¿Cómo expiras claves? — Con `EXPIRE / TTL` o fijando TTL al crear (`SET k v EX 60`). Redis usa expiración perezosa + muestreo activo.

3. Rate limiting en el API Gateway

P4 Explica cómo Spring Cloud Gateway hace rate limiting con Redis.

R El `RedisRateLimiter` implementa un **token bucket**: se configuran `replenishRate` (tokens/segundo), `burstCapacity` (máximo acumulable) y `requestedTokens` por petición. En cada request el gateway ejecuta un **script Lua atómico** en Redis que decrementa tokens y decide permitir (200) o rechazar (429). Lua garantiza atomicidad (leer + calcular + escribir en un solo paso), evitando condiciones de carrera entre réplicas del gateway.

R La clave se deriva de un `KeyResolver` (por IP, por usuario del JWT, o por API key). Ejemplo de configuración:

```
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
          uri: lb://auth-service
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
                key-resolver: "#{@ipKeyResolver}"
```

FOLLOW-UP ¿Por qué centralizar el rate limit en Redis y no en memoria del gateway? — Porque hay **varias réplicas** del gateway; un contador en memoria local no vería el tráfico de las otras instancias. Redis es el estado compartido y consistente.

P5 ¿Qué respondes al cliente cuando se supera el límite?

R **HTTP 429 Too Many Requests**, idealmente con cabeceras `X-RateLimit-Remaining`, `X-RateLimit-Burst-Capacity` y `Retry-After`. Esto protege especialmente el endpoint de login de `auth-service` frente a fuerza bruta.

4. Caché y patrones

P6 ¿Qué patrón de caché usarías para el catálogo de productos?

R **Cache-aside** (lazy loading): el servicio consulta Redis primero; si hay *miss*, lee de MongoDB, guarda en Redis con TTL y responde. Es simple y resiliente (si Redis cae, se degrada a la base). Alternativas: *write-through* (escribe en caché y BD a la vez, más consistente) y *write-behind* (asíncrono, más rápido pero con riesgo de pérdida). Para un catálogo mayormente de lectura, cache-aside con TTL corto es lo adecuado.

FOLLOW-UP ¿Cómo invalidas la caché cuando cambia el stock? — Publicando un evento (o borrando la clave) al actualizar el producto; TTL corto acota la ventana de datos obsoletos aunque falle la invalidación.

P7 Explica cache stampede, penetration y avalanche.

R

- **Stampede:** muchas peticiones fallan a la vez al expirar una clave popular y golpean la BD; se mitiga con *locks/single-flight* o recomputación anticipada.
- **Penetration:** se consultan claves inexistentes que nunca cachean; se mitiga cacheando el "no existe" (valor nulo con TTL corto) o un filtro de Bloom.
- **Avalanche:** muchas claves expiran simultáneamente; se mitiga con TTL con *jitter* aleatorio.

5. Persistencia y alta disponibilidad

P8 ¿RDB vs AOF? ¿Redis pierde datos?

R

RDB son snapshots periódicos (rápido de restaurar, compacto, pero puede perder los cambios desde el último snapshot). **AOF** registra cada escritura (más durable, con `fsync` configurable: `always/everysec/no`). Para caché pura la pérdida es tolerable; para datos importantes se usa AOF (o ambos). En este proyecto Redis es principalmente caché y rate limiting, donde perder estado no es crítico (se reconstruye).

P9 ¿Cómo escalas Redis y das alta disponibilidad?

R

Replicación primario-réplica para lecturas y failover; **Sentinel** para failover automático y descubrimiento; **Cluster** para *sharding* (particiona por hash slots, 16384 slots) cuando el dataset no cabe en un nodo. En Kubernetes se suele desplegar con un operador o Helm chart y un StatefulSet.

FOLLOW-UP ¿La replicación es síncrona? — No por defecto: es **asíncrona**, así que un failover puede perder las últimas escrituras no replicadas. `WAIT` permite semi-sincróna.

6. Diseño abierto / escenarios

P10 Se cae Redis. ¿Qué pasa con el gateway y el login?

R El `RequestRateLimiter` depende de Redis: hay que decidir la política de **fail-open** (permitir tráfico si Redis no responde, priorizando disponibilidad) o **fail-closed** (rechazar, priorizando protección). Para caché cache-aside, se degrada leyendo directo de la BD. Lo correcto es tener Redis en HA (Sentinel/Cluster) para que esta caída sea rara y breve.

P11 ¿Cómo implementas revocación de JWT con Redis?

R Manteniendo una **denylist**: al hacer logout o revocar, se guarda el `jti` (id del token) en Redis con TTL igual al tiempo restante de vida del token. El gateway/servicios comprueban si el `jti` está en la denylist antes de aceptarlo. El TTL asegura que la entrada se limpia sola cuando el token habría expirado igualmente.

Apéndice — Chuleta rápida

Tema	Punto clave
Rol en el POS	Rate limiting del gateway + caché (+ denylist JWT/sesiones)
Modelo	In-memory, single-threaded por comando → atómico y rápido
Rate limit	Token bucket vía script Lua atómico; estado compartido entre réplicas
Límite superado	HTTP 429 + <code>Retry-After</code> ; protege login de fuerza bruta
Caché	Cache-aside con TTL; cuidado con stampede/penetration/avalanche
Persistencia	RDB (snapshots) vs AOF (log de escrituras); replicación asíncrona
HA / escala	Sentinel (failover) + Cluster (sharding por hash slots)
Antipatrón	<code>KEYS *</code> bloquea; usar <code>SCAN</code>